

INTERNATIONAL ISLAMIC UNIVERSITY CHITTAGONG
Department of Computer Science & Engineering (CSE)
BSc in CSE, Final Examination, Autumn 2025

Course Code: **CSE-3527**
 Full Marks: **50**

Course Title: **Compiler**
 Time: **2 Hour 30 minutes**

[Answer all of the following questions from Group A and B.
 (Figures at right margin illustrate the marks (M), course objectives (COs), and bloom's taxonomy (DL))]

- | No. | Group – A [2×10 = 20 Marks] | M | COs | DL |
|-------|--|----------------|------------|--------|
| 1. a) | Syntax Analysis: Describe different type of syntax error that handling in C compiler construction and give the main error recovery strategies to handle these errors. | 2
(1+1) | CO3 | U |
| b) | Syntax Analysis: What is the impact of the left recursion for parsing in compiler construction? Is there any left recursion in the following grammar G_1 over the language $\Sigma = \{a, b, c, d\}$? If yes, eliminate the left recursion.
$A \rightarrow Ac \mid Aad \mid bd \mid \epsilon$ | 3
(1+2) | CO3 | An |
| c) | Consider the following grammar:
$S \rightarrow Bd \mid Cb$
$B \rightarrow aB \mid \epsilon$
$C \rightarrow cC \mid \epsilon$
i) Calculate the FIRST and FOLLOW sets for the following grammar
ii) Is the above grammar is LL(1) grammar? Justify your answer.
iii) Explain the significance of the "1" in LL(1). | 5 | CO3 | C |
| 2. a) | Consider the following Grammar:
$E \rightarrow E + T \mid T$ $T \rightarrow T * F \mid F$ $F \rightarrow (E) \mid id$
Show the shift-reduce parsing steps for the input string: $id + id * id$ | 2 | CO3 | U |
| Or, | What are the conditions for a grammar to be an operator precedence grammar? | 2 | CO3 | U |
| b) | Syntax Analysis:
Consider the following grammar G_4 over the alphabet $\Sigma = \{a, b, c, d\}$. You want to use an CLR(1) parser for the strings defined by G_4 . You only need to provide accepting states, including their items, and transitions between accepting states. | 8
(3+2+1+2) | CO3
CO3 | C
C |

$I_0: \begin{array}{l} S' \rightarrow .S, \$ \\ S \rightarrow .AAa, \$ \\ A \rightarrow .aA, a/b \\ A \rightarrow .bb, a/b \end{array}$	$I_0: \begin{array}{l} S' \rightarrow .S, \$ \\ S \rightarrow .Aa, \$ \\ A \rightarrow .bAc, \$ \\ A \rightarrow .dc, \$ \\ A \rightarrow .bda, \$ \\ A \rightarrow .d, a \end{array}$
---	--

Fig. 2(b)

Fig. 2(b) Or

- i) Provide the DFA transitions and state items of the LR(1) canonical collections. The first LR(1) DFA state item (I_0) already been provided in Fig. 2(b) for your reference.
- ii) Construct CLR(1) parser table. The parser table includes both the action and goto tables.
- iii) Is the grammar CLR(1)? Justify your answer.
- iv) Show the stack implementation along with the output parse tree for the input string 'abbbba'.
- Or,
- What do you mean by the conflict in a LR parsing table construction? Explain the possible types of conflict in a LR parsing and consider the following two grammar G_5 over the alphabet $\Sigma = \{a, b, c, d\}$.

$S \rightarrow Aa \mid bAc \mid dc \mid bda$ $A \rightarrow d$

Provide the DFA transitions and state items of the LR(1) canonical collections. The first LR(1) DFA state item (I_0) already been provided in Fig. 2(b) Or for your reference.

- i) Construct LALR(1) parser table. The parser table includes both the action and goto tables.
- ii) Show that above CFG G_5 is not SLR(1) grammar

Group – B [3×10 = 30 Marks]

3. a) Syntax directed translation: Write syntax-directed translation for the following grammar G_6 over the language $\Sigma = \{+, -, (,), id, num\}$ with every non-terminal has a synthesized attribute **.nptr** of pointer type for syntax tree constructions and write the basic functions to construct the syntax tree for arithmetic expression with example: 5 (2+1+2) CO4 An

$E \rightarrow E + T \mid E - T \mid T \quad T \rightarrow (E) \mid id \mid num$

Give the syntax-directed definition of the following grammar G_5 . Construct the syntax tree with instructions to define pointer for the input expression string "a+5-c+1"

- b) Type checking: 5 (2+1+2) CO4 Ap
- Write the basic examples of static checkers and show the placement of type-checker on compiler construction with a block diagram
 - What do you mean by "call by value" and "call by reference"
 - Write syntax-directed translation (semantic rules) for the following type checking statements syntax definitions:

$S \rightarrow \text{if } E \text{ then } S_1 \quad S \rightarrow \text{While } E \text{ do } S_1$

4. a) Run-time environment: 5 CO4 Ap
- What is the activation tree in the run-time environment of a compiler for a function?
 - What is the purpose of activation tree in the run-time environment of a compiler. Draw the activation tree for the following c programming function to generate the value of n^{th} factorial number, if $n=8$:

```
int factorial(int n){
    //Base case: if n is 0 or 1, return n
    if (n <= 1){
        return 1;
    }
    return n * factorial(n - 1);
}
```

- Explain the heap and stack allocation strategies for memory allocation

Or, What is the role of a **control stack**? Draw the activation tree for the following program: 5 CO4 Ap

```
main() {
    f();
}

f() {
    g();
    h();
}
```

- b) Intermediate code generation: Consider the following C program: 5 CO5 Ap

```
int main(){
    int a, b, c, d;
    scanf("%d%d%d", b,c,d);
    a=b*-c+b*-c+d; // '-' is unary minus
    print("%d", a);
    return 0; }
```

Translate the C assignment statement into:

- | | | |
|------------------------|----------------|--------|
| i) Syntax tree | iii) Triples | v) DAG |
| ii) Three-address code | iv) Quadruples | |

Or, Translate the following expression to quadruple, triple, indirect triple and polish notation: $a + b * c / e + b * c$ 5 CO5 Ap

5. a) Consider the following C program (Answer any one): 7 CO4 Ap

```
for (i=0; i<n; i++)
    for (j=0; j<n; j++)
        c[i][j] = 0.0;
for (i=0; i<n; i++)
    for (j=0; j<n; j++)
        for (k=0; k<n; k++)
            c[i][j] = c[i][j] + a[i][k]*b[k][j];
```

Translate the above C program into three address code. Also construct basic block and flow graph for the above C code.

- b) Draw the syntax tree and DAG for the following expression: $a+a+(a+a+a+(a+a+a+a))$ 3 CO5 An

International Islamic University Chittagong

Department of Computer Science & Engineering

B.Sc. in CSE, Final Examination, Spring 2025

Course Code: CSE-3527

Course Title: Compiler

Total Marks: 50

Time: 2 Hours 30 Minutes

[Answer *all* the following questions. Figures in the right hand margin indicate full marks. Use a Separate answer script for Group-A and Group-B.]

Group A

1. a) The following is a grammar for regular expressions over symbols a and b only, using $+$ in place of $|$ for union, to avoid conflict with the use of vertical bar as a meta symbol in grammars:

$\text{rexpr} \rightarrow \text{rexpr} + \text{rterm} \mid \text{rterm}$

$\text{rterm} \rightarrow \text{rterm} \text{rfactor} \mid \text{rfactor}$

$\text{rfactor} \rightarrow \text{rfactor} * \mid \text{rprimary}$

$\text{rprimary} \rightarrow a \mid b$

a) Left factor this grammar.

b) Does left factoring make the grammar suitable for top-down parsing?

c) In addition to left factoring, eliminate left recursion from the original grammar.

d) Is the resulting grammar suitable for top-down parsing?

Marks COs

5 3

- b) Compute FIRST and FOLLOW for the following grammar:

$E \rightarrow TE'$

$E' \rightarrow +TE' \mid \epsilon$

$T \rightarrow FT'$

$T' \rightarrow *FT' \mid \epsilon$

$F \rightarrow (E) \mid \text{id}$

5 3

2. a) Consider the following grammar:

$S \rightarrow AA$

$A \rightarrow aA \mid b$

i. Construct the augmented grammar for the given grammar.

ii. Derive the SLR(1) items for the grammar and then construct the parsing table.

5 3

- b) For the grammar $S \rightarrow 0S1 \mid 01$, indicate the handle of string "000111" using Shift-Reduce parsing.

OR

2. a) Draw a hierarchy diagram to differentiate the all parsers in compiler construction and also draw the block diagram of LR parser.

2 3

- b) Consider the following grammar G_1 over the alphabet $\Sigma = \{a, b, c, d\}$. You want to use an SLR(1) or CLR(1) parser for the strings defined by G_1 . You only need to provide accepting states, including their items, and transitions between accepting states.

$S \rightarrow aBa \mid AB \mid d$

$A \rightarrow b \mid c$

$B \rightarrow a$

8 3

- i) Draw the DFA transitions and state items of the LR(0) item collections. The first LR(0) DFA state item (I_0) already been provided in Fig. 2 for your reference.
- ii) Construct SLR(1) parsing table from LR(0) DFA state items. The parser table includes both the action and goto sections.
- iii) Is the grammar SLR? Justify your answer.
- iv) Show the stack implementation along with the output parse tree for the input string "ca".

$S \rightarrow S$
$S \rightarrow .a5a$
$S \rightarrow .45$
$S \rightarrow .d$
$A \rightarrow .b$
$A \rightarrow .c$

Fig. 2

Group B

3. a) A Syntax-directed definition of a simple desk calculator is given below:

Sl. No.	PRODUCTION	SEMANTIC RULES
1.	$L \rightarrow E n$	$L.val = E.val$
2.	$E \rightarrow E_1 + T$	$E.val = T.val + T_1.val$
3.	$E \rightarrow T$	$E.val = T.val$
4.	$T \rightarrow T_1 * F$	$T.val = T_1.val \times F.val$
5.	$T \rightarrow F$	$T.val = F.val$
6.	$F \rightarrow (E)$	$F.val = E.val$
7.	$F \rightarrow digit$	$F.val = digit.lexval$

Give annotated parse tree and dependency graph for the above expression:
 $(3 + 4) + (5 + 6) n$.

- b) Step by step construct syntax tree using the following grammar and string "a-4-c":

$E_1 \rightarrow E_1 + T$
 $E_1 \rightarrow E_1 - T$
 $E_1 \rightarrow T$
 $T \rightarrow (E_1)$
 $T \rightarrow id$
 $T \rightarrow num$

OR

3. a) Syntax directed translation:

Write the functions to construct the syntax tree for arithmetic expression. Consider the following logical operations CFG G_0 over the language $\Sigma = \{not, or, and, true, false\}$ with every non-terminal has a synthesized attribute val of Boolean type:

$B \rightarrow B \text{ and } C \mid B \text{ or } C \mid C$

$C \rightarrow not \ C \mid true \mid false$

Give the syntax-directed definition of the following grammar G_s . Draw the annotated parse for the input expression string "not true or false and true"

- b) Type checking:

- Write the purposes of the type-checker on compiler?
- Explain with the semantic rules of type checking for C programming expressions. statements and function in compiler type checker.
- Write semantic rules for the following type checking expressions:

$E \rightarrow E_1 \text{ op } E_2$

$E \rightarrow E_1 [E_2]$

Run-time environment:

5 4

i) What is the activation tree in the run-time environment of a compiler for a function?

ii) Draw the activation tree for the following c programming function to generate the n^{th} Fibonacci number, if $n=7$:

```
int Fibonacci(int n){
    //Base case: if n is 0 or 1, return n
    if (n <= 1){
        return n;
    }
    return Fibonacci(n - 1) + Fibonacci(n - 2);
}
```

iii) Explain the memory allocation strategies for the run-time environment.

Intermediate code generation:

Translate the arithmetic expression $x + x * (y - z) + (y - z) * w$ into:

i) Syntax tree

iv) DAG

ii) Three-address code

v) Quadruples

iii) Triples

a) Code generation and optimization:

Consider the following C codes and answer the questions (i) to (iii):

```
void bubbleSort(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}
```

i) Translate the C programming code into three-address code.

ii) Identify the basic block in three-address code.

Construct the flow graph from the three-address code.

b) Code optimization:

i) What are the structure preserving transformation of basic blocks? Explain one of them with appropriate example for code optimization.

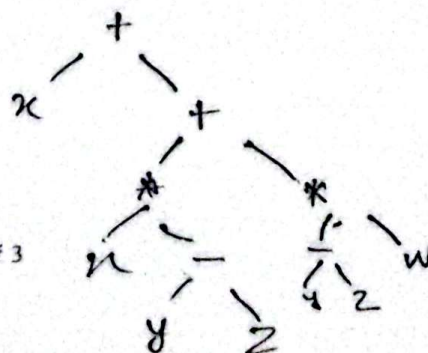
ii) Construct the DAG for the following basic block.

$w := x * y$

$x := w - z$

$y := x * y$

$z := w - z$



International Islamic University Chittagong

Department of Computer Science and Engineering

Final Term Examination, Spring-2024

Course Title: Compiler

Course Code: CSE-3527

Full Marks: 50

Time: 2 Hours 30 Minutes

Answer all Five of the following questions.
Figures in the right hand margin indicate full marks.

Group A

1. Write the role of a parser. Draw a hierarchy diagram to show all types of common parsers used in compiler construction. 2 CO2
 - a) What is the rule of left recursion in compiler? Is there any left recursion in the following grammar G_1 ? If yes, Eliminate the left recursion. 3 CO3
$$\begin{aligned} S &\rightarrow S - A | A \\ B &\rightarrow B + C | C \\ C &\rightarrow id \end{aligned}$$
 - c) Consider the following CFG G_2 over the language $\Sigma = \{int, (,), +, *\}$. 5 CO2
$$\begin{aligned} E &\rightarrow TX \\ T &\rightarrow int Y | (E) \\ X &\rightarrow +E | \epsilon \\ Y &\rightarrow *T | \epsilon \end{aligned}$$
 - i) Find out the FIRST and FOLLOW set for each of the non-terminals in the above grammar
 - ii) Construct the predictive parsing table based on the above grammar G_2 and show the stack implementation along with output parse tree for the input string " $int*int$ ". Is this grammar LL (1)? Explain.
2. What is the basic difference of LR (0), SLR(1), CLR(1) and LALR parser tables? Why '\$' symbol uses as an element of follow set of the start variable of a grammar in the parser? 2 CO3
- b) Consider the following grammar G_2 over the alphabet $\Sigma = \{1, -, *\}$. You want to use an SLR(1) or CLR(1) parser to parse strings defined by G_2 . Notice that each production in G_2 is associated with a production number in first bracket. 8 CO3

$$(1) S \rightarrow N * N$$

$$(2) S \rightarrow N * P$$

$$(3) N \rightarrow -P$$

$$(4) P \rightarrow 1P$$

$$(5) P \rightarrow 1$$

I_0 :

$S' \rightarrow .S$
$S \rightarrow .N * N$
$S \rightarrow .N * P$
$N \rightarrow .-P$

Fig. 2(a)

I_0 :

$S' \rightarrow .S, \$$
$S \rightarrow .N * N, \$$
$S \rightarrow .N * P, \$$
$N \rightarrow .-P, *$

Fig. 2(b)

- OR**
- a) Construct the LR (0) parsing table for the following grammar:
- $$S \rightarrow \Lambda \Lambda$$
- $$\Lambda \rightarrow a\Lambda/b$$
- b) Parse the following string using the parsing table you just made in 2(a).
- abab\$
- c) Give bottom-up parsing for the input string aa*a+ for the following grammar:
- $$S \rightarrow SS+ | SS* | a$$

3 CO2

5 CO3

2 CO3

Group B

- 3.
- a) What is the basic difference between synthesized and inherited attributes for syntax-directed translation? Consider the following CFG G3 over the language $\Sigma = \{\text{int, float, char, double, id, ', '}\}$.

5 CO2

$$D \rightarrow TL$$

$$T \rightarrow \text{int} | \text{float} | \text{char} | \text{double}$$

$$L \rightarrow L_1, \text{id} | \text{id}$$

Write semantic rules for syntax-directed translation of the above grammar G3. Construct the dependency graph for the input string "double a, b, c".

- b) Write the impact of the type-checker on compiler construction? Write semantic rules for the following type checking expressions and statements:

5 CO3

$$E \rightarrow E_1 \text{mod} E_2$$

$$E \rightarrow E_1 \text{op} E_2$$

$$S \rightarrow \text{if } E \text{ then } S_1$$

$$S \rightarrow \text{while } E \text{ } S_1$$

4.

- a) What is the activation tree in the run-time environment of a compiler for a function? Explain the memory allocation strategies for the run-time environment. Draw the activation tree for the following c programming function if n=6:

CO3

```
long factorial (int n) {
    if (n==1)
        return 1;
    else
        return n*factorial(n-1);
}
```

- b) Translate the arithmetic expression $o := x * (-y) + x * (-y) + x * y$ into

5

CO3

- i) Syntax tree iv) DAG
- ii) Three-address code v) Quadruples
- iii) Triples

5.
a) Consider the following C codes and answer the questions (i) to (iii):

5 CO3

```
void insertionSort(int array[], int n){
    int i, key, j;
    for (i = 1; i < n; i++){
        key = array[i];
        j = i - 1;
        while (j >= 0 && array[j] > key){
            array[j+1] = array[j];
            j=j-1;
        }
        array[j+1] = key;
    }
}
```

- i) Translate the C code into three-address code.
ii) Identify the basic block in three-address code.
iii) Construct the flow graph from the three-address code.
- b) Write some code optimization techniques used in compiler construction with examples and Construct the dag for the following basic block.

5 CO3

```
d := b * c
e := a + b
b := b * c
a := e - d
```

OR

- a) Consider the following C program (Answer any one):

7 CO2

```
(i) for I from 1 to 10 do
    for j from 1 to 10 do
        a[i, j] = 0.0;
    for I from 1 to 10 do
        a[i, j] = 1.0;
```

```
(ii) for (i=0; i<n; i++)
    for (j=0; j<n; j++)
        c[i][j] = 0.0;
    for (i=0; i<n; i++)
        for (j=0; j<n; j++)
            for (k=0; k<n; k++)
                c[i][j] = c[i][j] + a[i][k]*b[k][j];
```

Translate the above C program into three address code. Also construct basic block and flow graph for the above C code.

- b) Draw the syntax tree and DAG for the following expression:
a+a+(a+a+a+(a+a+a+a))

3 CO2

International Islamic University Chittagong

Department of Computer Science and Engineering

B. Sc. in CSE Final Term Examination, Autumn-2023

Course Code: CSE-3527 Course Title: Compiler

Total marks: 50 Time: 2.5 Hours

[Answer all the questions, Figures in the right hand margin indicate full marks]

CO

Group A

1. a) Construct CLR(1) parsing table for the following grammar: A 4 CO2
 $S \rightarrow Aa \mid bAc \mid dc \mid bda$ P
 $A \rightarrow a$
- b) If the following grammar is suitable for LL (1) parsing then check whether the string $qqqqr\$$ can be generated from the grammar by showing step by step operations. A 6 CO3
P
 $A \rightarrow Q$
 $Q \rightarrow St \mid Rr$
 $S \rightarrow sS \mid \epsilon$
 $R \rightarrow qR \mid \epsilon$
2. a) Consider the following grammar: A 6 CO3
P
 $X \rightarrow X + Y \mid Y$
 $Y \rightarrow Y d F \mid F$
 $F \rightarrow (X) \mid id$
(i) Find the first and follow functions for the given CFG with Left Recursive production rules.
(ii) Is the above grammar is LL(1) grammar? Answer the question with proper justification.
- b) Consider the following grammar: A 4 CO3
P
 $E \rightarrow T E'$
 $E' \rightarrow + T E' \mid \square$
 $T \rightarrow F T'$
 $T' \rightarrow * F T' \mid \square$
 $F \rightarrow (E) \mid id$
Show the sequence of moves on input $id*id+id$ using the nonrecursive predictive parser algorithm.
- OR,
2. a) Consider the grammar: A 4 CO3
P
 $S \rightarrow (L) \mid a$
 $L \rightarrow L, S \mid S$
1. Construct a right most derivation for $(a, (a, a))$ parse tree.
2. Show the steps of shift reduce parser considering right most derivation for $(a, (a, a))$
- b) Consider the following grammar: A 6 CO3
P
 $S \rightarrow MM$
 $M \rightarrow mM \mid n$
1. Construct the simple LR parsing table by using canonical items
2. Explain the parsing strategies with stack implementation for input string "mmnn"

Group B

3. a) Define the Synthesized attributes and inherited attributes. Using the following production rules and semantic rules draw the annotated parse tree and dependency graph for the sentence float id1, id2, id3.

Ap 1+ CO3
5

PRODUCTION	SEMANTIC RULES
1) $D \rightarrow T L$	$L.inh = T.type$
2) $T \rightarrow \text{int}$	$T.type = \text{integer}$
3) $T \rightarrow \text{float}$	$T.type = \text{float}$
4) $L \rightarrow L_1, \text{id}$	$L_1.inh = L.inh$ $\text{addType}(\text{id.entry}, L.inh)$
5) $L \rightarrow \text{id}$	$\text{addType}(\text{id.entry}, L.inh)$

- b) Mention the differences among parse tree, syntax tree and DAG. U 2 CO2
- c) Write the algorithm for Left factoring. R 2 CO3
4. a) Explain two strategies for dynamic storage allocation. R 4 CO1
- b) By using SDD of simple desk calculator, construct annotated parse trees for the following expression: $3*5 + 4n$ U 4 CO1
- c) Define DAG. Draw the DAG for this basic block: U 2 CO3
- $a = b + c$
 $b = b - d$
 $c = c + d$
 $e = b + c$
5. a) Consider the following C code and answer the questions (i) to (iii) Ap 5 CO3
- ```

Void production(int a[], int b[]){
 int prod=0, i=1;
 do{ prod=prod + a[i]*b[i];
 i=i+1;
 }while(i<=20);
 printf("product=%d", prod);
}

```
- i) Translate the C code into 3 address code
- ii) Identify the basic block in 3 address code
- iii) Construct the flow graph from the 3-address code
- b) Define basic block and flow graph with example. U 3 CO1
- c) Fill in the blanks: R 2 CO1
- i) Data type grammar use \_\_\_\_ attributes
- ii) Symbol table is generated in \_\_\_\_ phase.
- iii) Operator precedence parser is \_\_\_\_ parser
- iv) \_\_\_\_ parser can parse using ambiguous grammar.
- OR,
5. a) Translate the following expression to quadruple, triple and indirect triple- Ap 5 CO3
- $a + b * c / e + b * c$
- b) What do you mean by Peephole optimization? Write down the U 3 CO1
- characteristics of Peephole optimization with example.
- c) Explain the primary tasks of code generator. R 2 CO1



# International Islamic University Chittagong

Department of Computer Science & Engineering

B.Sc. in CSE, Final Exam, Spring 2023

Course Code: CSE-3527 Course Title: Compiler

Total Marks: 50 Time: 2.5 Hr.

Figures in the right-hand margin indicate full marks

## Group A

1. a. Define left factoring with equation. Write down the algorithm of Left Recursion. M CO DI  
 b. Write down the role of parser. 3 CO3 C3  
 Do left factoring for the following grammar: 3

$A \rightarrow aAB \mid aA \mid a$

$B \rightarrow bB \mid b$

- c. Consider the grammar to find the First and Follow and parsing table: 4

$S \rightarrow B d \mid C b$

$B \rightarrow a B \mid \epsilon$

$C \rightarrow cC \mid \epsilon$

OR,

- Consider the grammar to find the First and Follow and parsing table:

$S \rightarrow iEtss' \mid a$

$S' \rightarrow \epsilon \mid eS$

$E \rightarrow b$

2. a. Eliminate left recursion for the following grammar: 4 CO3 C3

i.  $E \rightarrow E + T \mid T$

ii.  $T \rightarrow T * F \mid F$

iii.  $F \rightarrow (E) \mid id$

iv.  $S \rightarrow (L) \mid a \mid b$

$L \rightarrow L, S \mid S$

- b. Determine if the following grammar is suitable for LL (1) parsing, then check whether the string "qqqqr" can be generated from the grammar by showing step by step operations. 6

$A \rightarrow Q$

$Q \rightarrow St \mid Rr$

$S \rightarrow sS \mid \epsilon$

$R \rightarrow qR \mid \epsilon$

OR,

- Create an Operator Precedence Table for the following grammar:

$A \rightarrow A+A \mid A*A \mid a$

- Using the Operator Precedence Table justify the following string by showing step by step operations:  $a + a * a * a + a * a$

## Group B

3. a. Define synthesized attribute and inherited attribute. 2 CO3 C2  
 b. Draw the syntax tree and DAG for the expression 4  
 $(a*b) + (c-d) * (a*b) + b$

OR,

$((a + b) * (c - d)) + ((e - f) * (g + h))$

- c. Considering the following declaration 4

$D \rightarrow TL$

$T \rightarrow int \mid real$

$L \rightarrow L1, id \mid id$

- i. Give semantic rule.

- ii. Draw a noted parse tree with inherited attributes and dependency graph for the sentence `real id1, id2, id3` by considering grammar.

- 4 a. Write the algorithm for portioning three address instructions into basic blocks 2 CO3  
 b. Derive the 3-address code, Quadruples, Triples and Indirect Triples for the following 6  
 statement: `a := b * - c + b * - c`

OR,

Derive the 3-address code, Quadruples, Triples and Indirect Triples for the following statement: `z := x * (-y) + x * (-y) + x * y`

- c. Show the position of code generator in the compiler process. 2

- 5 a. Consider the following C code and answer the question (i) to (iii): 7 CO1 C

```
int dec = 0, i = 0, count=0;
while (n!=0) {
 rem = n % 10;
 n /= 10;
 dec += (rem * 2)/i;
 ++i;
}
do {
 n /= 10;
 ++count;
} while (n != 0);
```

- i. Translate the C code into three address code  
 ii. Identify the basic block in three address code  
 iii. Construct the flow graph from the three-address code

- b. Write the differences between Recursive Predictive Descent Parser and Non-Recursive Predictive Descent Parser 3

# International Islamic University Chittagong

Department of Computer Science and Engineering

B. Sc. in CSE

Final Exam, Spring-2022

Course Code: CSE 3527

Time: 2 hours 30 minutes

Course Title: Compiler

Full Marks: 50

- (i) The figures in the right-hand margin indicate full marks  
(ii) Course Outcomes and Bloom's Levels are mentioned in additional Columns

## Part A

[Answer the questions from the followings]

1. a) Define LL(1) grammar and LR(k) parser. Why LR parsers are used in compiler design?  
CLO 2 R 3
1. b) Construct CLR(1) and LALR(1) data flow diagram and parsing table for the following grammar:  
 $S \rightarrow CC$   
 $C \rightarrow cC$   
 $C \rightarrow d$   
CLO 2 U 7
2. a) Calculate the LR(0) Parsing for the given grammar-  
 $E \rightarrow E + T \mid T$   
 $T \rightarrow T * F \mid F$   
 $F \rightarrow (E) \mid id$   
CLO 3 App 5
2. b) What do you mean by Operator Precedence Grammar?  
With the help of following grammar parse the input string:- "id + id \* id" \$  
 $T \rightarrow T + T \mid T * T \mid id$   
CLO 2 E 1+4  
=5
- Or,
2. a) For the following sub-problems, consider the following context-free grammar:  
CLO 3 App 1+4  
=5
- $S \rightarrow AS$  (1)  
 $A \rightarrow xAx$  (2)  
 $A \rightarrow C$  (3)  
 $B \rightarrow yBy$  (4)  
 $B \rightarrow C$  (5)  
 $C \rightarrow zBz$  (6)  
 $C \rightarrow wAw$  (7)  
 $C \rightarrow \lambda$  (8)

i. What are the terminals and non-terminals of this grammar?

ii. Show the derivation of the string  $xxxx\$$  starting from  $S$  (specify which production you used at each step), and give the parse tree according to that derivation.

2 b) Construct a CLR parsing table for the given context-free grammar

CLO 2 E 5

$S \rightarrow AA$

$A \rightarrow aA \mid b$

### Part B

[Answer the questions from the followings]

3. a) Define the synthesized attributes and inherited attributes with example. Using syntax-directed definition draw the annotated parse tree and dependency graph for the sentence  $(3*5+2)*2n$ .

CLO 1 C 5

$S \rightarrow En$

$E \rightarrow E + T$

$E \rightarrow T$

$T \rightarrow T * F$

$T \rightarrow F$

$F \rightarrow (E)$

$F \rightarrow \text{digit}$

3. b) Write some examples of top-down parsing and bottom-up parsing. Consider the following grammar-

CLO 2 E 1+4  
=5

$E \rightarrow E - E$

$E \rightarrow E \times E$

$E \rightarrow \text{id}$

Parse the input string  $\text{id} - \text{id} \times \text{id}$  using a shift-reduce parser.

4. a) What is code optimization? Consider the following code:

CLO 3 App 1+4  
=5

$a = b * -c + b * -c$

Write down the Quadruples of the given Code?

4. b) Write quadruple, triples and indirect triples for following expression :

CLO 2 U 5

$(x + y) * (y + z) + (x + y + z)$

5. a)  $t1 = 4 * I$   
 $t2 = a[t1]$   
 $t3 = 4 * I$   
 $t4 = b[t3]$   
 $t5 = t2 * t4$   
 $t6 = \text{prod} + t5$   
 $t7 = I + 1$   
 $I = t7$   
If  $I \leq 20$  goto 1

CLO 3 An 5

Write down the DAG to get the above information.

- b) What do you know about the identifying the Basic blocks in target code generation?  
What do you know about the Peep hole optimization?

CLO 2 An 3+2=5

Or,

5. a) Consider the following C codes and answer the questions (i) to (iii):-

CLO 3 An 5

```
int a[100], i, n=10, steps=0, temp;
do {i=steps+1
 do {if(a[steps]>a[i]){
 temp=a[steps];
 a[steps]=a[i];
 data[i]=temp;
 i++;
 }while(steps<n);
 steps++;
}while(steps<n);
```

- (i) Translate the C code into three-address code.  
(ii) Identify the basic block in three-address code.  
(iii) Construct the flow graph from the three-address code.

- b) Draw the syntax tree and DAG for the expression:

CLO 2 An 5

$a+a+a(a*a)+a+a+a*(b-c)+(b-c)*d$